



FACULTÉ DES SCIENCES DHAR EL MAHRAZ FÈS

UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH DE
FÈS

Rapport

Intitulé : Grey Wolf algorithm

Réalisé par :

ARBAOUI OTHMANE
BERCHID WALID
BOUKHRIS ABDELHAMID

Introduction

Au cours des deux dernières décennies, les techniques d'optimisation métaheuristiques ont gagné une popularité considérable. Cette montée en notoriété soulève la question fondamentale de comprendre les raisons qui ont conduit à leur adoption généralisée. La réponse à cette interrogation peut être synthétisée en quatre motivations principales : simplicité, flexibilité, mécanisme sans dérivation et capacité à éviter les optima locaux. En premier lieu, les métaheuristiques se distinguent par leur simplicité, tirant leur inspiration de concepts fondamentaux, souvent liés à des phénomènes physiques, des comportements animaux ou des principes évolutifs. Cette simplicité offre aux informaticiens la possibilité de simuler divers concepts naturels, de concevoir de nouvelles métaheuristiques, de fusionner différentes approches, ou encore d'améliorer celles déjà existantes.

Deuxièmement, la flexibilité caractérise la capacité des métaheuristiques à s'appliquer à une diversité de problèmes sans nécessiter des modifications substantielles dans la structure de l'algorithme. Ces techniques sont aisément adaptables à des problèmes variés, les considérant souvent comme des boîtes noires, où seules les entrées et sorties du système revêtent une importance pour la métaheuristique.

Troisièmement, la plupart des métaheuristiques opèrent avec des mécanismes sans dérivation. À la différence des approches d'optimisation basées sur les gradients, les métaheuristiques optimisent de manière stochastique, débutant le processus d'optimisation avec des solutions aléatoires. Cette caractéristique les rend particulièrement adaptées aux problèmes réels où les informations dérivées sont coûteuses ou inconnues.

Enfin, les métaheuristiques se démarquent par leur capacité accrue à éviter les optima locaux, comparativement aux techniques d'optimisation conventionnelles. Cette aptitude découle de la nature stochastique des métaheuristiques, qui leur permet d'éviter la stagnation dans les solutions locales et d'explorer de manière approfondie l'espace de recherche.

Chapitre 1 : La méthode de GWO

Description

La méthode d'optimisation du loup gris (*Grey Wolf Optimization*, GWO) représente une technique intelligente d'essaim élaborée par Mirjalili et al. en 2014, s'inspirant de la hiérarchie de leadership observée chez les loups, reconnus pour leur mode de chasse en groupe.

Ce procédé algorithmique imite le comportement social de chasse des loups gris dans leur habitat naturel. Dans cette approche, la population est segmentée en quatre groupes distincts : alpha (α), bêta (β), delta (δ) et oméga (ω).

Les trois loups les plus robustes sont désignés comme α , β et δ , assumant le rôle de guides pour orienter les autres membres du groupe (ω) vers des zones prometteuses au sein de l'espace de recherche.

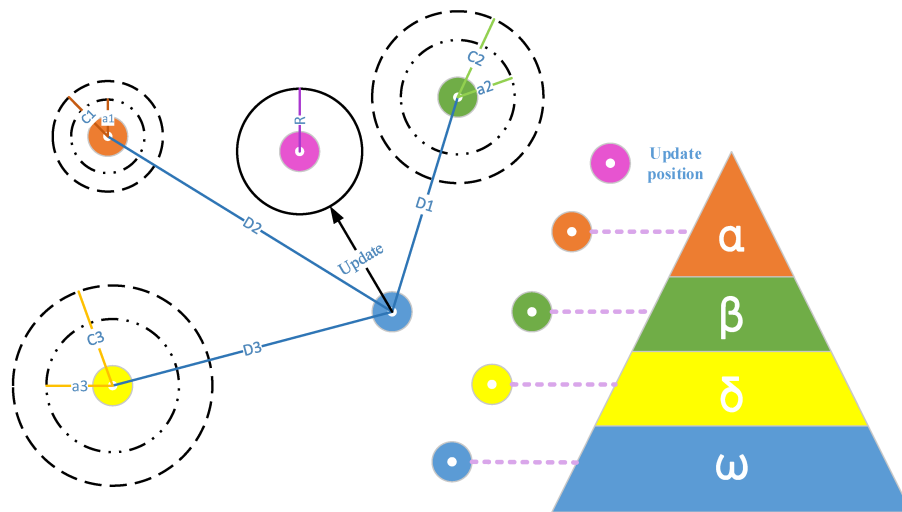


FIGURE 1 – Structure hiérarchique du loup gris (la dominance décroît de la partie supérieure à la partie inférieure)

Source d'inspiration

Les loups gris sont reconnus en tant que prédateurs occupant le sommet de la chaîne alimentaire et sont connus pour leur propension à vivre en groupe, généralement composé de 5 à 12 individus en moyenne.

D'une importance particulière est leur hiérarchie sociale stricte, où le premier niveau est attribué aux alphas (α). Ces loups alpha, constitués d'un mâle et d'une femelle, jouent le rôle de leaders au sein de la meute, prenant des décisions cruciales telles que la stratégie de chasse, le lieu de repos, et d'autres aspects. Bien que les ordres de l'alpha soient prépondérants, des comportements démocratiques peuvent également émerger, avec un alpha suivant parfois les autres loups du groupe. L'alpha, également désigné comme le loup dominant, a le privilège exclusif de l'accouplement au sein de la meute. Il est notable que l'alpha ne doit pas nécessairement être le loup le plus fort physiquement, mais plutôt celui qui excelle dans la gestion du groupe, soulignant ainsi l'importance de l'organisation et de la discipline au sein de la meute.

Le deuxième niveau est attribué aux bêtas (β), qui sont des loups subordonnés assurant un rôle de soutien à l'alpha dans la prise de décisions et d'autres activités de la meute. Les bêtas, qu'ils soient mâles ou femelles, sont susceptibles de devenir les alphas en cas de décès ou de vieillissement des loups alpha. Les bêtas, tout en respectant les alphas, exercent également un certain niveau d'autorité sur les loups de niveaux inférieurs, jouant le rôle de conseillers et de disciplinaires au sein du groupe. Ils renforcent les ordres des alphas et fournissent des retours d'information.

Le troisième niveau est dénommé Delta (δ), représentant les loups subordonnés qui doivent se soumettre aux alphas et aux bêtas. Ils dominent cependant les omégas, le niveau le plus bas. Cette catégorie comprend des rôles tels que les scouts, sentinelles, anciens, chasseurs, et gardiens. Les scouts sont chargés de surveiller les limites territoriales et d'alerter la meute en cas de danger, les sentinelles assurent la protection et la sécurité du groupe. Les anciens sont des loups expérimentés ayant occupé précédemment des rôles d'alpha ou de bêta. Les chasseurs aident les alphas et les bêtas lors de la chasse et contribuent à la fourniture de nourriture pour la meute.

Le quatrième niveau (le plus bas) est appelé oméga (ω). L'oméga joue le rôle de bouc émissaire et doit se soumettre à tous les autres loups dominants. Ils sont les derniers autorisés à manger, bien que leur rôle puisse sembler moins significatif, la perte de l'oméga peut entraîner des problèmes au sein de l'ensemble de la meute. Cela est dû à la libération de la violence et de la frustration par l'oméga, contribuant

ainsi à maintenir la stabilité de la structure de dominance et à satisfaire le groupe dans son ensemble.

En plus de la hiérarchie sociale des loups, l'algorithme GWO intègre également les comportements sociaux liés à la chasse de groupe des loups gris. Ces comportements, tels que le suivi, la poursuite, l'encerclement, le harcèlement, et enfin, l'attaque de la proie, sont modélisés dans l'algorithme GWO.



FIGURE 2 – Comportement de chasse des loups gris : (A) traque, chasse et approche de la proie (B-D) poursuite, encerclement et harcèlement de la proie. (E) état stationnaire et attaque.

Modèle mathématique et algorithme

Dans cette sous-section, les modèles mathématiques de la hiérarchie sociale, de la poursuite, de l'encerclement et de l'attaque de la proie sont fournis. Ensuite, l'algorithme GWO est exposé.

Hiérarchie sociale

Afin de modéliser mathématiquement la hiérarchie sociale des loups lors de la conception de GWO, nous considérons la solution la plus adaptée comme l'alpha (α). Par conséquent, les deuxième et troisième meilleures solutions sont appelées bêta (β) et delta (δ) respectivement. Le reste des solutions candidates est supposé être omega (ω). Dans l'algorithme GWO, la chasse (optimisation) est guidée par α , β et δ . Les loups suivent ces trois loups.

Encerclement de la proie

Comme mentionné précédemment, les loups gris encerclent leur proie lors de la chasse. Afin de modéliser mathématiquement ce comportement d'encerclement, les équations suivantes sont proposées :

$$\vec{D} = |\vec{C} \cdot \vec{X}_p - \vec{X}(t)| \quad (\text{II.6})$$

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A}\vec{D} \quad (\text{II.7})$$

Où t indique l'itération actuelle, $\vec{A}$ et $\vec{C}$ sont des vecteurs de coefficients, $\vec{D}$ est le résultat du calcul, $\vec{X}_p$ est le vecteur de position de la proie, et $\vec{X}(t)$ indique le vecteur de position d'un loup gris. Les vecteurs $\vec{A}$ et $\vec{C}$ sont calculés de la manière suivante :

$$\vec{A} = 2\vec{a} \cdot \vec{r}_1 \quad (\text{II.8})$$

$$\vec{C} = 2 \cdot \vec{r}_2 \cdot \vec{X}_p \quad (\text{II.9})$$

Les composants de $\vec{a}$ diminuent linéairement de 2 à 0 au fil des itérations, et $\vec{r}_1, \vec{r}_2$

sont des vecteurs aléatoires dans $[0, 1]$. Pour observer les effets des équations (II.6) et (II.7), un vecteur de position bidimensionnel et certains des voisins possibles sont illustrés dans la Fig. 3 (a). Comme le montre cette figure, un loup gris à la position (X, Y) peut mettre à jour sa position en fonction de la position de la proie (X^*, Y^*) . Différents endroits autour du meilleur agent peuvent être atteints par rapport à la position actuelle en ajustant les vecteurs $\vec{A}$ et $\vec{C}$. Par exemple, $(X^* - X, Y^*)$ peut être atteint en définissant $\vec{A} = (1, 0)$ et $\vec{C} = (1, 1)$. Les positions mises à jour possibles d'un loup gris dans l'espace tridimensionnel sont représentées dans la Figure 3 (b). Notez que les vecteurs aléatoires $\vec{r}_1$ et $\vec{r}_2$ permettent aux loups d'atteindre n'importe quelle position entre les points illustrés dans la Fig. 3. Ainsi, un loup gris peut mettre à jour sa position à l'intérieur de l'espace autour de la proie dans n'importe quel emplacement aléatoire en utilisant les équations (II.6) et (II.7).

Le même concept peut être étendu à un espace de recherche à n dimensions, et les loups gris se déplaceront en hyper-cubes (ou hyper-sphères) autour de la meilleure solution obtenue jusqu'à présent.

Chasse

Les loups gris ont la capacité de reconnaître l'emplacement de leurs proies et de les encercler. La chasse est généralement guidée par l'alpha. Le bêta et le delta peuvent

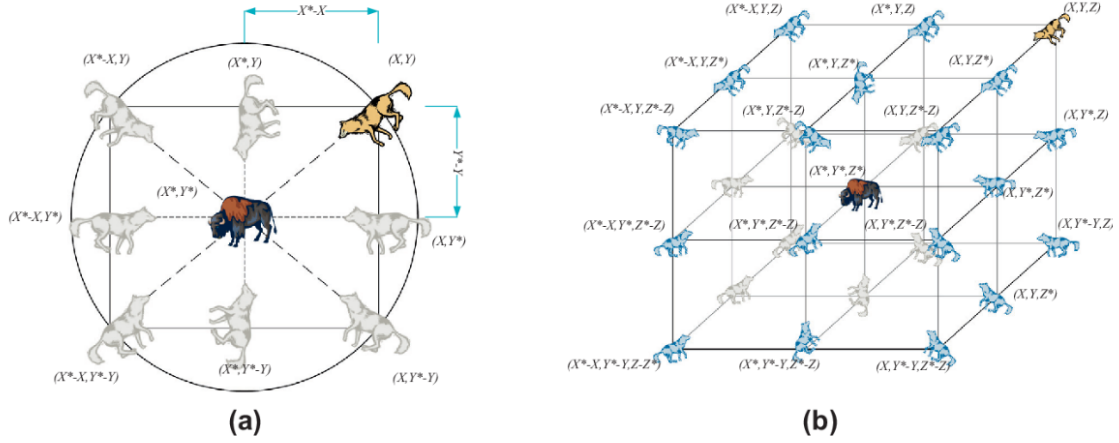


FIGURE 3 – Vecteurs de position 2D et 3D et leurs prochains emplacements possibles

également participer à la chasse de manière occasionnelle. Cependant, dans un espace de recherche abstrait, nous n'avons aucune idée de l'emplacement de l'optimum (proie). Afin de simuler mathématiquement le comportement de chasse des loups gris, nous supposons que l'alpha (meilleure solution candidate), le bêta et le delta ont une meilleure connaissance de l'emplacement potentiel de la proie. Par conséquent, nous sauvegardons les trois premières meilleures solutions obtenues jusqu'à présent et obligeons les autres agents de recherche (y compris les omegas) à mettre à jour leurs positions en fonction de la position du meilleur agent de recherche.

Les formules suivantes sont proposées à cet égard :

$$\begin{cases} \vec{D}_\alpha = |\vec{C}_1 \cdot \vec{X}_\alpha - \vec{X}| \\ \vec{D}_\beta = |\vec{C}_2 \cdot \vec{X}_\beta - \vec{X}| \\ \vec{D}_\delta = |\vec{C}_3 \cdot \vec{X}_\delta - \vec{X}| \end{cases} \quad (\text{II.10})$$

Où :

- $\vec{X}_\alpha(t)$ représente la position de l'alpha.
- $\vec{X}_\beta(t)$ indique la position de beta.
- $\vec{X}_\delta(t)$ est la position de delta.
- $\vec{C}_1, \vec{C}_2, \vec{C}_3$ sont des vecteurs aléatoires, et $\vec{X}$ indique la position de la solution actuelle.

$$\begin{cases} \vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 \cdot \vec{D}_\alpha \\ \vec{X}_2 = \vec{X}_\beta - \vec{A}_2 \cdot \vec{D}_\beta \\ \vec{X}_3 = \vec{X}_\delta - \vec{A}_3 \cdot \vec{D}_\delta \end{cases} \quad (\text{II.11})$$

$$\vec{X}(t+1) = \frac{\vec{x}_1 + \vec{x}_2 + \vec{x}_3}{3} \quad (\text{II.12})$$

Algorithme

Algorithm 1 Grey Wolf Optimization Algorithm

- 1: Initialize the grey wolf population X_i ($i = 1, 2, \dots, n$)
 - 2: Initialize a , A , and C
 - 3: Calculate the fitness of each search agent
 - 4: X_α = the best search agent
 - 5: X_β = the second best search agent
 - 6: X_δ = the third best search agent
 - 7: **while** ($t < \text{Max number of iterations}$) **do**
 - 8: **for** each search agent **do**
 - 9: Update the position of the current search agent by equation (3.7)
 - 10: **end for**
 - 11: Update a , A , and C
 - 12: Calculate the fitness of all search agents
 - 13: Update X_α , X_β , and X_δ
 - 14: $t = t + 1$
 - 15: **end while**
 - 16: **return** X_α
-

Chapitre 1 : Utilisation de GWO sur le Problème du Voyageur de Commerce (TSP)

Introduction

Dans ce chapitre, nous explorerons l'application de l'optimisation par le loup gris (GWO) pour résoudre le Problème du Voyageur de Commerce (TSP). Le TSP est un problème NP-complet dans le domaine de l'optimisation combinatoire, nécessitant la recherche du chemin le plus court qui visite chaque ville exactement une fois et revient au point de départ. L'algorithme GWO, inspiré du comportement social des loups gris, offre une approche prometteuse pour résoudre ce problème de manière efficace.

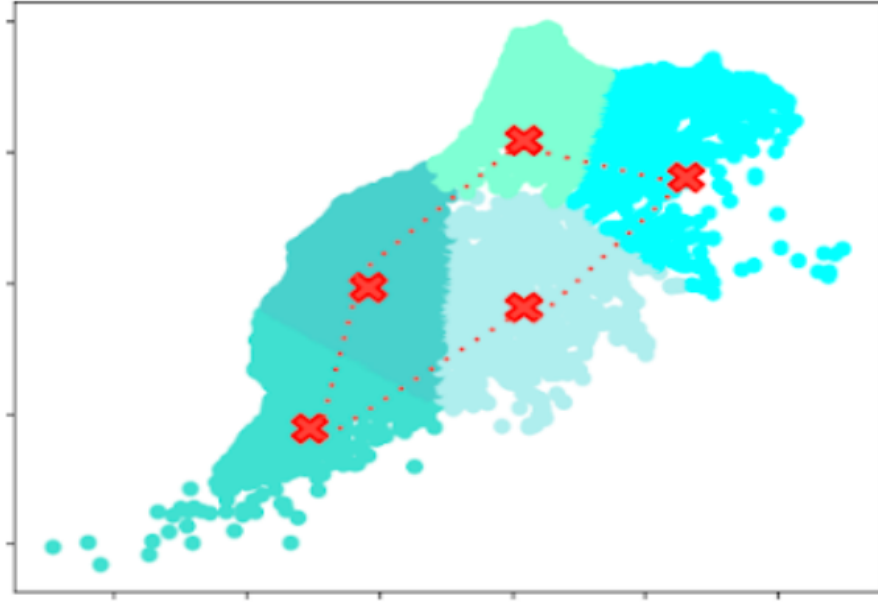


FIGURE 4 – le Problème du Voyageur de Commerce Au Maroc

Formalisation du Problème du Voyageur de Commerce

Avant d'explorer l'application de GWO, il est essentiel de formaliser le Problème du Voyageur de Commerce. Considérons un ensemble de n villes et une matrice de distances d représentant les distances entre chaque paire de villes. L'objectif est de trouver la permutation optimale des villes minimisant la distance totale parcourue.

Présentation de l'Optimisation par le Loup Gris (GWO)

Comportement Social des Loups Gris

Le GWO s'inspire du comportement social des loups gris, où chaque loup joue un rôle crucial. Trois rôles principaux sont définis : *Alpha*, *Beta*, et *Delta*, correspondant respectivement au loup dominant, au loup suivant en termes de fitness, et au troisième loup en termes de fitness.

Formules Mathématiques de l'Algorithme GWO

L'algorithme GWO utilise des formules mathématiques pour mettre à jour les positions des loups. Pour chaque loup i , les mises à jour des positions Alpha, Beta, Delta et de la meute sont calculées à l'aide des formules suivantes :

Mise à Jour de la Position Alpha :

$$D_1 = |C \cdot \text{fitness}(\alpha) - \text{fitness}(\text{Wolf}_i)|$$

$$X_1 = \text{Wolf}_i - C \cdot D_1$$

Mise à Jour de la Position Beta :

$$D_2 = |C \cdot \text{fitness}(\beta) - \text{fitness}(\text{Wolf}_i)|$$

$$X_2 = \text{Wolf}_i - C \cdot D_2$$

Mise à Jour de la Position Delta :

$$D_3 = |C \cdot \text{fitness}(\delta) - \text{fitness}(\text{Wolf}_i)|$$

$$X_3 = \text{Wolf}_i - C \cdot D_3$$

Mise à Jour des Positions de la Meute :

$$\text{Wolf}_i = \frac{X_1 + X_2 + X_3}{3}$$

Adaptation de GWO au Problème du Voyageur de Commerce

Pour adapter GWO au TSP, nous définissons la représentation des solutions comme des permutations des villes. La population initiale est générée avec des permutations aléatoires, et la fitness est évaluée en fonction de la distance totale de la route.

Itération Complète de l'Algorithme GWO (TSP)

Initialisation (Début de l'algorithme)

- Générer une population initiale de loups avec des routes aléatoires.
- Calculer la fitness de chaque loup en évaluant la distance totale de sa route.

Sélection d'Alpha, Beta et Delta

- Identifier les trois loups avec les plus basses fitness comme alpha, beta et delta.

Mise à Jour des Positions (pour chaque loup i)

Mise à Jour de la Position Alpha :

$$D_1 = |C \cdot \text{fitness}(\alpha) - \text{fitness}(\text{Wolf}_i)|$$

$$X_1 = \text{Wolf}_i - C \cdot D_1$$

Mise à Jour de la Position Beta :

$$D_2 = |C \cdot \text{fitness}(\beta) - \text{fitness}(\text{Wolf}_i)|$$

$$X_2 = \text{Wolf}_i - C \cdot D_2$$

Mise à Jour de la Position Delta :

$$D_3 = |C \cdot \text{fitness}(\delta) - \text{fitness}(\text{Wolf}_i)|$$

$$X_3 = \text{Wolf}_i - C \cdot D_3$$

Mise à Jour des Positions de la Meute :

$$\text{Wolf}_i = \frac{X_1 + X_2 + X_3}{3}$$

Mise à Jour de la Population

- Comparer les nouvelles fitness avec les anciennes pour chaque loup.
- Si la nouvelle fitness est meilleure, mettre à jour la position du loup.

Sélection d'Alpha, Beta et Delta (Mise à Jour)

- Identifier les trois loups avec les plus basses fitness parmi les positions mises à jour comme alpha, beta et delta.

Fin de l'Iteration 1.

Ce Qu'il Faut Faire Après la Première Itération

Répétition du Processus

- Répéter les étapes de mise à jour des positions, de mise à jour de la population et de sélection d'alpha, beta et delta pour un nombre spécifié d'itérations (par exemple, 50 itérations).

Convergence

- Surveiller la convergence de l'algorithme. Si les solutions convergent vers une solution optimale, l'algorithme peut être arrêté prématurément.

Évaluation des Résultats

- Après les itérations, évaluer les résultats finaux. La meilleure route trouvée et la distance totale parcourue peuvent être extraites.

Analyse et Interprétation

- Analyser les résultats obtenus. Comparer la performance de l'algorithme GWO pour le TSP avec d'autres approches.

Perspectives Futures

- Identifier les domaines d'amélioration et les perspectives futures. L'algorithme peut être ajusté, et d'autres méthodes peuvent être explorées pour obtenir de meilleures performances.

Conclusion de l'Application de GWO au TSP

- Conclure la section en résumant les résultats, en discutant des avantages et des limitations, et en soulignant l'efficacité de l'algorithme pour résoudre le TSP.

Explications du Code Python

La fonction `initialize_population` crée une population initiale de trajets aléatoires.

La fonction `calculate_fitness` mesure la distance totale d'un trajet basé sur la matrice des distances.

La fonction `FullTour` vérifie si un trajet couvre toutes les villes.

L'algorithme `GWO_TSP` utilise l'optimisation par le loup gris pour trouver le meilleur trajet.

```

import numpy as np

def initialize_population(population_size, num_cities):
    # Initialize the population with random tours
    population = [np.random.permutation(range(num_cities)) for _ in range(population_size)]
    return population

def calculate_fitness(tour, distance_matrix):
    # Calculate the total distance of the tour based on the distance matrix
    total_distance = 0
    for i in range(len(tour)):
        total_distance += distance_matrix[tour[i - 1]][tour[i]] # Fix indexing issue
    return total_distance

def FullTour(tour, num_cities):
    return len(set(tour)) == num_cities

def GWO_TSP(num_cities, population_size, max_iterations, distance_matrix):
    Population = initialize_population(population_size, num_cities)

    for iteration in range(max_iterations):
        fitness_values = [calculate_fitness(tour, distance_matrix) for tour in Population]

        max_fitness_index = np.argmax(fitness_values)
        Population.pop(max_fitness_index)

        if FullTour(Population[0], num_cities):
            break

        for i in range(population_size):
            X_alpha = min(Population, key=lambda x: calculate_fitness(x, distance_matrix))
            X_beta = min([x for x in Population if not np.array_equal(x, X_alpha)], key=lambda x:
calculate_fitness(x, distance_matrix))
            X_delta = min([x for x in Population if not np.array_equal(x, X_alpha) and not
np.array_equal(x, X_beta)], key=lambda x: calculate_fitness(x, distance_matrix))

            Population[i] = (Population[i] + X_alpha) % num_cities # Fix indexing issue
            Population[(i + 1) % population_size] = (Population[(i + 1) % population_size] + X_beta) %
num_cities # Fix indexing issue
            Population[(i + 2) % population_size] = (Population[(i + 2) % population_size] + X_delta) %
num_cities # Fix indexing issue

        best_solution = min(Population, key=lambda x: calculate_fitness(x, distance_matrix))
        return best_solution

moroccan_cities = ["Casablanca", "Marrakech", "Rabat", "Fes", "Tangier"]
num_cities = len(moroccan_cities)
population_size = 5
max_iterations = 100

distance_matrix = np.array([[0, 200, 90, 300, 400],
                             [200, 0, 250, 150, 350],
                             [90, 250, 0, 180, 280],
                             [300, 150, 180, 0, 200],
                             [400, 350, 280, 200, 0]])

best_solution = GWO_TSP(num_cities, population_size, max_iterations, distance_matrix)

print("Best solution (city names):", [moroccan_cities[i] for i in best_solution])
print("Total distance:", calculate_fitness(best_solution, distance_matrix))

```

FIGURE 5 – Représentation visuelle de l'algorithme GWO pour le PVC.

n est le nombre total d'observations dans l'ensemble de données, Y est la valeur réelle de la variable dépendante pour l'observation i $\hat{Y}$ est la valeur prédite par le modèle pour l'observation i

Chapitre 2 : Utilisation de GWO sur les Réseaux de Neurones

Utilisation de GWO sur les Poids du Réseau de Neurones

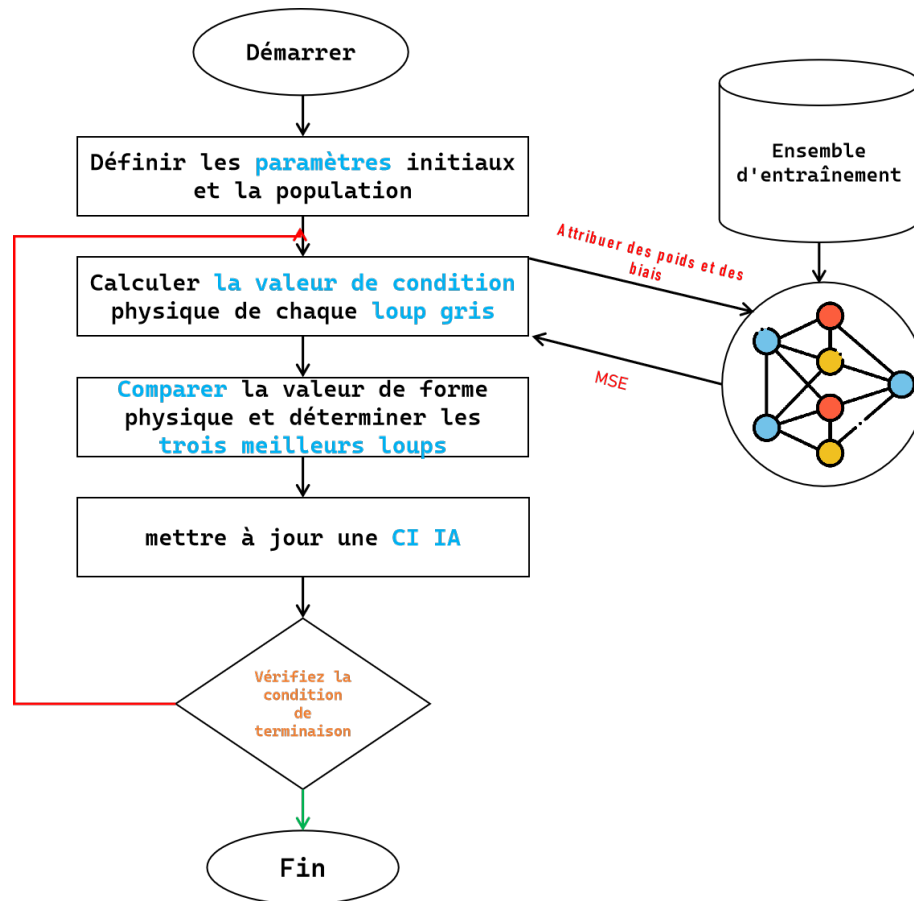


FIGURE 6 – Schéma d’Optimisation des paramètres pour un Réseau Neuronal Efficace

L’algorithme GWO considère les poids et les biais comme une population, chaque loup de cette population représente les poids et les biais d’un neurone, et les ajuste pour optimiser le taux d’erreur et obtenir les meilleures performances lors des étapes d’entraînement et de test. La première étape du modèle proposé consiste à initialiser de manière aléatoire des poids et des biais en tant que population ou meute de loups.

La deuxième étape consiste à calculer l'erreur pour chaque loup en utilisant l'équation de l'erreur quadratique moyenne (MSE) afin de distinguer les meilleurs et les pires paramètres. Ensuite, les trois meilleures solutions sont marquées comme alpha, beta, delta respectivement.

L'étape suivante consiste à ajuster la position du reste des loups (omega). Ces étapes sont répétées jusqu'à ce que la condition d'arrêt soit satisfaite. Enfin, la position d'alpha est stockée comme la solution optimale globale

erreur quadratique moyenne (MSE)

Erreur quadratique moyenne :

$$EQM = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

FIGURE 7 – erreur quadratique moyenne (MSE) formule

Utilisation de GWO sur les Hyperparamètres du Réseau de Neurones

Pour obtenir un réseau neuronal efficace, on peut ajouter un algorithme qui choisit les bons hyperparamètres pour notre réseau neuronal. Gwo est une solution parmi d'autres pour accomplir cette tâche. Nous avons implémenté un algorithme pour choisir le nombre de couches cachées à ajouter à notre réseau neuronal, ainsi que le nombre de neurones dans chaque couche et le taux d'apprentissage pour obtenir le meilleur réseau neuronal qui donne une bonne précision lors de la phase de test.

Le diagramme suivant expliquera la méthode de notre travail :

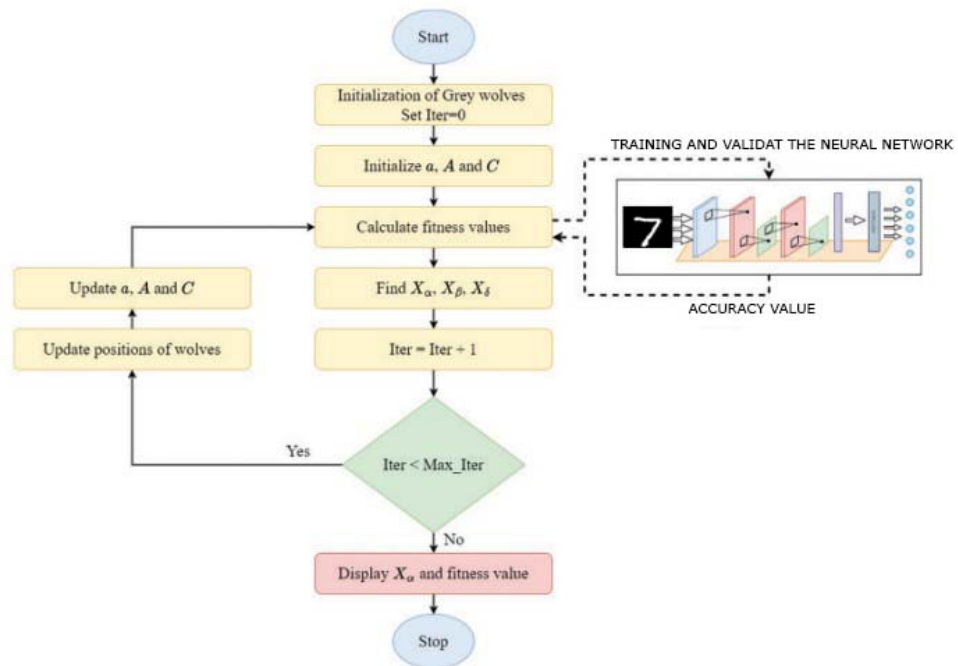


FIGURE 8 – Schéma d’Optimisation des Hyperparamètres pour un Réseau Neuronal Efficace

```

import numpy as np
import random
from keras.datasets import mnist
from keras.utils import to_categorical

import sys
sys.path.append('F:\WISD\RO\GW0_project')

from GW0_Neural_network.neural_network import Layer , neural_network

class GWOptimazer:
    def __init__(self, max_iter):
        self.max_iter = max_iter

    def calcul_fitness(self, position, x_train, y_train, x_test, y_test):
        net = neural_network()
        net.addLayer(Layer(28*28, position[1].astype(int)))
        for _ in range(position[0].astype(int)):
            net.addLayer(Layer(position[1].astype(int), position[1].astype(int)))
        net.addLayer(Layer(position[1].astype(int), 10))

        net.fit(x_train[0:100], y_train[0:10000], 10, position[2])

        predicted_value = np.array(net.predict(x_test[0:10]))
        predicted_value = predicted_value.astype('float32')
        true_values = np.array(y_test[0:10])
        return net.calculate_accuracy(predicted_value, true_values)

    def optimizer(self, x_train, y_train, x_test, y_test):
        iter = 0
        num_wolf = 4
        dimension = 3

        min_bound = [3, 16, 0.0001]
        max_bound = [9, 256, 0.1]

        position = np.random.uniform(min_bound, max_bound, size=(num_wolf, dimension))
        position_int = position
        fitness_value = np.zeros(num_wolf)

        for i in range(position_int.shape[0]):
            fitness_value[i] = self.calcul_fitness(position_int[i], x_train, y_train, x_test, y_test)

        a = 2*(1-0/self.max_iter)
        A = 2*random.random()*a - a
        C = 2*random.random()

        while( iter < self.max_iter):

            fitness_value_sorted = np.sort(fitness_value)[::-1]

            alpha = np.where(fitness_value == fitness_value_sorted[0])[0][0]
            beta = np.where(fitness_value == fitness_value_sorted[1])[0][0]
            delta = np.where(fitness_value == fitness_value_sorted[2])[0][0]

            x_alpha = position_int[alpha]
            x_beta = position_int[beta]
            x_delta = position_int[delta]

            for wolf in range(num_wolf):

                x = position_int[wolf]

                D_alpha = abs(C*x_alpha-x)
                D_beta = abs(C*x_beta-x)
                D_delta = abs(C*x_delta-x)

                x1 = abs(x_alpha-A*D_alpha)
                x2 = abs(x_beta-A*D_beta)
                x3 = abs(x_delta-A*D_delta)

                new_x = ((x1+x2+x3)/3)
                new_x = new_x.reshape(1, dimension)

                if new_x[0, 0] > 12:
                    new_x[0, 0] = 12
                elif new_x[0, 0] < 1:
                    new_x[0, 0] = 1

                if new_x[0, 1] < 16:
                    new_x[0, 1] = 16
                elif new_x[0, 1] > 256:
                    new_x[0, 1] = 256

                new_fitness_value = self.calcul_fitness(new_x[0], x_train, y_train, x_test, y_test)

                if(new_fitness_value > fitness_value[wolf]):
                    position_int[wolf] = new_x
                    fitness_value[wolf] = new_fitness_value

            iter = iter + 1

            a = 2*(1-0/self.max_iter)
            A = 2*random.random()*a - a
            C = 2*random.random()

        return x_alpha, fitness_value[alpha]

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(x_train.shape[0], 1, 28*28)
x_train = x_train.astype('float32')
x_train /= 255

y_train = to_categorical(y_train)

x_test = x_test.reshape(x_test.shape[0], 1, 28*28)
x_test = x_test.astype('float32')
x_test /= 255
y_test = to_categorical(y_test)

GW0 = GWOptimazer(3)
hyperparam.my_fitness_value = GW0.optimizer(x_train, y_train, x_test, y_test)
print('the best hyperparameters :')
print('hidden layer : ', hyperparam[0].astype(int))
print('every layer have : ', hyperparam[1].astype(int))
print('the learning rate : ', hyperparam[2])
print('the accuracy of this hyperparameters :', my_fitness_value)

```

Références

- [1] Seyedali Mirjalili, Seyed Mohammad Mirjalili, Andrew Lewis, *Grey wolf optimizer*, *Advances in engineering software*, **69**, 46–61, Elsevier, 2014.
- [2] Saman M Almufti, Hawar B Ahmad, Ridwan B Marqas, Renas R Asaad, *Grey wolf optimizer : Overview, modifications and applications*, *International Research Journal of Science, Technology, Education, and Management*, **1**(1), 1–1, 2021.
- [3] Karuna Panwar, Kusum Deep, *Discrete Grey Wolf Optimizer for symmetric travelling salesman problem*, *Applied Soft Computing*, **105**, 107298, Elsevier, 2021.